



마이크로프로젝트 2024. 08 - 2025.10

발급매니저 앱 서비스 2025. 06 - 2025. 10

- 참여인원 : BackEnd : 3명 / FrntEnd 3명 / 기획 : 1명 / 디자인 : 1명 / 총 : 8명
- 담당업무 : 아키텍처 전환 과정에서 이벤트 구조 설계, Outbox 모델링, Polling 처리 로직 구현 및 API 개발
- 사용기술 : Kotlin, Exposed, Spring Boot, PostgreSQL, Kafka, AWS(S3, ECS, MSK)

[개요]

대리인이 위임장을 통해 병원 서류를 대신 발급받고 서류를 앱으로 업로드하여 보험 청구를 진행할 수 있는 기능을 설계 및 구현했습니다.

기존 모놀리식 구조에서 기능 장애가 전체 서비스로 확산되는 문제를 해결하기 위해 Strangler 패턴을 적용하여 청구 도메인을 점진적으로 분리했습니다.

Kafka 기반 이벤트 드리븐 아키텍처를 도입하고 Outbox 패턴을 적용하여 데이터 원자성과 서비스 간 결합도를 개선했습니다.

또한 이벤트 배치 처리 구조로 개선하여 배당된 작업 1개 당 약 3600건의 이벤트를 800ms로 처리했습니다.

[문제]

- 모놀리식 구조로 인해 특정 기능 장애가 전체 서비스로 확산
- 일부 기능 수정 시 전체 서비스 배포 필요 → 배포 리스크 증가
- 동기 호출 구조로 인해 서비스 간 결합도가 높고 확장성 제한

[해결 방안]

- Strangler 패턴을 적용하여 청구 도메인을 점진적으로 분리
- Kafka 기반 이벤트 드리븐 아키텍처 도입으로 비동기 통신 구조 전환
- 이벤트 발행의 원자성 보장을 위해 Outbox 패턴 적용
- 폴링 기반 이벤트 발행 구조를 설계하여 애플리케이션 레벨에서 제어 가능하도록 구성

[문제 분석 및 해결]

이벤트 발행의 원자성 문제

- DB 트랜잭션과 Kafka 전송이 분리되어 데이터 불일치 발생
 - Outbox 테이블을 도입하여 서비스 로직과 이벤트 데이터를 하나의 트랜잭션으로 처리
 - 이후 Outbox를 기반으로 이벤트를 발행하여 데이터 일관성 확보

이벤트 발행 방식으로 Polling 선택

- CDC 기반 방식은 DB 설정 및 운영 리스크가 있다고 판단하여 추후에 도입 고려
 - 초기 단계에서는 안정성과 제어 가능성을 우선하여 Polling 방식 선택
 - 애플리케이션 레벨에서 제어 및 장애 대응이 용이하도록 설계
 - 실시간성보다 안정성을 우선하여 구조 결정

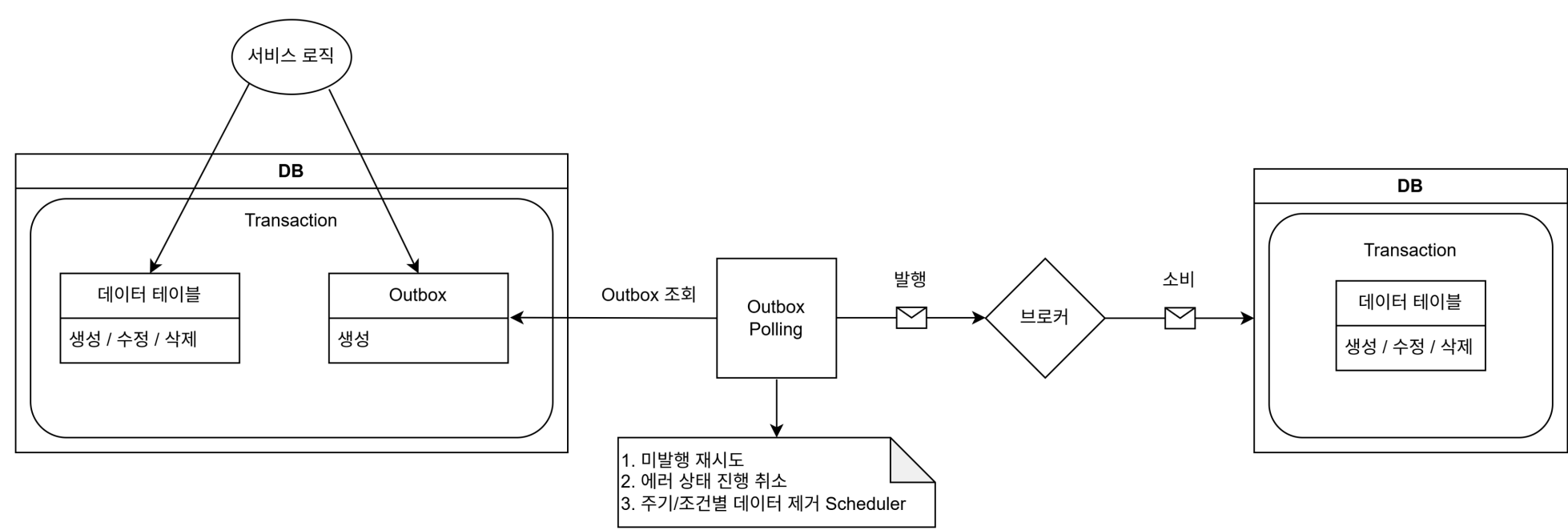
분산 환경에서 중복 및 순서 문제

- 다수 인스턴스에서 폴링 시 이벤트 중복 발행 가능
- 파티션 키 기반 순서 보장 어려움
 - FOR UPDATE SKIP LOCKED를 활용하여 중복 처리 방지 및 병렬 처리 구현
 - 파티션 키 기준으로 이벤트를 조회하여 동일 키는 하나의 인스턴스에서 순차 처리하도록 설계

[성과]

- 배치 발행 구조로 설계하여 약 3600건 기준 800ms로 처리
- 서비스 장애가 전체 시스템으로 확산되지 않도록 구조 개선
- 기능 단위 배포가 가능해져 배포 리스크 감소
- 트래픽이 집중되는 청구 도메인을 독립적으로 점진적 확장 가능하도록 구조 개선

[구조]



보험 상담 보장 분석 2025. 04 - 2025. 05

- 참여인원 : BackEnd : 1명 / FrnotEnd 1명 / 기획 : 1명 / 디자인 : 1명 / 총 : 4명
- 담당업무 : 보장 분석 스크래핑, 스크래핑 서버 구성, DB 설계
- 사용기술 : Kotlin, Jooq, Spring Boot, PostgreSQL, AWS(EC2)

[개요]

보험 보장 분석 웹에 분산된 데이터를 상담원이 직접 조회해야 하는 비효율 문제를 해결하기 위해 스크래핑 기반 데이터 수집 및 백오피스 조회 기능을 설계 및 구현

[문제]

- 외부 보장 분석 웹에서 API 미제공 및 네트워크 분석 불가로 인해 데이터 자동 수집 불가능
- 상담원이 직접 웹에 접속하여 정보를 확인해야 하는 비효율적인 업무 구조 존재

[해결]

- Kotlin + Selenium(headless) 기반 스크래핑 서버 구축하여 데이터 자동 수집
- 수집 데이터를 DB에 저장하여 백오피스에서 즉시 조회 가능하도록 기능 개발
- Firefox + GeckoDriver 조합을 선택하여 ChromeDriver 대비 버전 호환성 문제 최소화 및 운영 안정성 확보

[문제 분석 및 해결]

스크래핑 시 페이지 로딩 과정에서 메모리 스파이크로 인한 Out Of Memory 발생

- 서버 스펙 업 대신 SWAP 2GiB 적용으로 비용 증가 없이 문제 해결
- t4g.small 환경 유지하면서 안정적으로 스크래핑 처리 가능하도록 개선

Selenium 기반 크롤링에서 AJAX 로딩 및 SPA 특성으로 인해 발생하는 DOM 지연 렌더링과 문제

- 동적 페이지 로딩 및 DOM 변경에 대응하기 위해 WebDriverWait 기반 명시적 대기 및 예외 처리 로직 구성
- 재시도 로직을 구현하여 데이터 누락 없이 안정적인 수집 구조를 구축

[성과]

- 스크래핑 처리 시간 10,000ms → 7,000ms (30% 개선)
- 상담원의 수동 조회 과정 제거로 업무 효율성 개선
- 외부 의존적인 데이터 조회 구조를 내부 시스템으로 전환하여 운영 안정성 확보
- 추가 인프라 비용 없이 메모리 문제 해결

보험 상품 데이터 수집 및 처리 파이프라인 구축 2020. 11 - 2023. 07

- 참여인원 : BackEnd : 5명
- 담당업무 : 데이터 수집/처리 구조 설계, 공통 모듈화, 데이터 이력 관리, BackEnd 및 FrontEnd 개발
- 사용기술 : Java, MyBatis, Spring Boot, MySQL, JavaScript, JSP, JQuery, AWS(EC2)

[개요]

40여 개 보험사의 다이렉트 및 대면 상품 데이터를 수집

보험 비교 서비스에서 활용할 수 있도록 데이터 처리 파이프라인을 설계 및 구현

하루 평균 10만 건 이상의 데이터를 처리하는 환경에서 안정적인 데이터 수집과 정합성 확보

수집된 데이터를 서비스에서 활용 가능한 형태로 가공 및 저장하는 구조 구축

반복적인 스크래핑 로직을 공통화하여 유지보수성을 개선하고 데이터 변경 이력을 관리할 수 있도록 아카이빙 구조를 도입

[문제]

- 데이터 수집 환경에서 에러 발생 시 원인 파악 어려움
- 스크래핑 로직이 보험사별로 분산되어 코드 중복 및 유지보수성 저하
- API 호출 누락, 중복 호출, 예외 미처리 등 비정상 케이스 발생
- 보험 상품 개정 여부를 수동으로 확인하여 누락 및 불필요 작업 발생
- 데이터 변경 이력을 관리할 수 없어 정합성 검증 어려움

[해결]

- ErrorCode 기반 에러 로깅 구조를 설계하여 주요 예외를 선별 저장
- 스크래핑 종료 후 API 호출 구간 및 반복 로직을 공통 모듈로 분리
- 데이터 변경 이력 관리를 위해 아카이빙 테이블 구조 설계
- 작업 테이블과 아카이빙 데이터를 비교하여 변경 사항을 감지하도록 구성

[문제 분석 및 해결]

데이터 수집 환경에서의 에러 추적 문제

- 하루 평균 10만 건 이상의 데이터 처리 중 에러 발생 시 로그 파일 기반으로는 원인 파악이 어려움
 - ErrorCode Enum을 정의하여 주요 에러 발생 구간에 구조화된 로그를 저장
 - 이를 통해 백오피스에서 실시간으로 에러 현황을 확인할 수 있도록 개선하고 대응 시간 단축

중복 코드 및 유지보수성 저하 문제

- 보험사별로 개별 구현된 로직으로 인해 코드 중복 및 품질 편차 발생
 - 스크래핑 종료 후 데이터 저장 API 호출 구간을 공통화하여 호출 누락 및 중복 문제 방지
 - 클릭, 스크롤, 대기 처리 등 반복 동작을 공통 컴포넌트로 분리하여 재사용 구조로 개선
 - 페이지 구조 변경 시 공통 모듈을 사용해 코드 수정을 최소화하여 유지보수 비용 감소 및 작업 효율 향상

데이터 변경 감지 및 이력 관리 문제

- 보험 상품 개정 여부를 수동으로 확인하여 누락 및 불필요한 재처리 발생
 - 아카이빙 테이블을 도입하여 기존 데이터를 이력으로 저장하고 최신 데이터로 갱신
 - 작업 테이블과 아카이빙 데이터를 비교하여 변경 여부를 자동으로 식별하도록 구현
 - 데이터 변경 감지 자동화 및 운영 효율성 향상

[성과]

- 하루 평균 10만 건 이상의 데이터 수집 및 처리 파이프라인 안정화
- 에러 로그 구조 개선으로 장애 대응 시간 단축
- 공통 모듈화를 통해 유지보수 비용 절감 및 개발 효율 향상
- 데이터 이력 관리 및 변경 감지 자동화로 운영 효율성 개선

Career

스크래핑 성능 개선

- 참여인원 : BackEnd : 2명 / 총 : 2명
- 담당업무 : 데이터 관리, BackEnd 개발
- 사용기술 : Java, MyBatis, Spring Boot, MySQL, AWS(EC2)

[개요]

백오피스 서비스와 사용자 서비스에서 공통으로 사용되던 데이터 조회 구조를 분리하고 비효율적인 쿼리를 개선하여 API 응답 속도 및 데이터 처리 성능 향상
레거시로 누적된 비효율 쿼리를 리팩토링하고 데이터 접근 패턴에 맞는 최적화 전략을 적용하여 전체 시스템의 성능과 안정성을 개선

[문제]

- 백오피스와 사용자 서비스에서 동일한 쿼리를 사용하여 불필요한 데이터까지 조회되는 구조
- 비효율적인 JOIN, 서브쿼리, 와일드카드 등으로 인한 쿼리 성능 저하
- 레거시 쿼리 누적으로 인해 수정 시 영향 범위 파악이 어려운 구조
- 동시 접근 시 테이블 락 경합으로 인한 성능 저하 및 데이터 충돌 발생

[해결]

데이터 조회 구조 분리

- 백오피스와 사용자 서비스의 조회 목적에 맞게 쿼리를 분리하여 불필요 데이터 조회 제거
- 데이터 처리량 감소 및 응답 속도 개선

쿼리 성능 최적화

- 보험료 조회 쿼리 분리 및 인덱스 활용 **3,700ms → 170ms (약 95% 개선)**
- UNION ALL 제거 및 조회 구조 단순화 **2,000ms → 1,200ms**
- 서브쿼리를 JOIN으로 변경 **1,300ms → 900ms**
- DISTINCT 제거 → **1,500ms → 400ms**
- SELECT 컬럼 최소화 (약 20개 컬럼 제거) → **불필요 I/O 감소**

동시성 제어 구조 개선

- updated_at 기반 낙관적 락을 적용하여 충돌 감지 및 데이터 정합성 확보
- 특정 케이스에 한해 비관적 락 적용

[성과]

주요 API 응답 속도 최대 약 **95% 개선 (3,700ms → 170ms)**

불필요한 데이터 조회 제거로 DB 부하 감소 및 전체 처리 성능 향상

쿼리 구조 단순화로 유지보수성 개선 및 레거시 코드 제거

낙관적 락 & 비관적 락으로 데이터 정합성 확보 및 장애 가능성 감소